DAgger vs symmetric sampling in Super Tux Kart

Théo Fontaine
Théophile Lahaussois

Sorbonne Université – M2 MIND – Reinforcement Learning

Code: https://github.com/theolahaussois/DAgger-vs-symmetric-sampling-in-Super-Tux-Kart

Abstract

We compare DAgger and the *symmetric sampling* of RLPD for teaching a single SuperTuxKart kart to complete one lap as fast as possible, starting from the same hand-written expert and the same demonstration lap (609 transitions). Over 10 seeds per method, DAgger reaches the expert's level from its very first iteration and finishes all its laps; it is the best-performing and most reliable method for an equal budget (IQM of 102.5 versus 100.8 for the expert). RLPD starts from a random actor, completes its first lap after about 8 600 steps, then catches up with DAgger in median, but two seeds out of ten collapse. Without the demonstrations, the same algorithm never finishes a lap: symmetric sampling is what makes the problem solvable. Finally, both methods sometimes beat the expert by almost 3 s by avoiding a banana that the expert hits on every lap.

1 Introduction

To learn to drive from an expert driver π^*, one can *imitate* it, by asking which action it would have taken in the situations we encounter: this is DAgger [9]. One can also *learn by reinforcement* while reusing its trajectories as if they were past experience: this is the *symmetric sampling* of RLPD [2]. Both methods start from the same expert, but do not extract the same information from it: DAgger queries it online and ignores the reward; RLPD never queries it, but directly optimizes the reward. We compare them on a single SuperTuxKart kart that must complete one lap as fast as possible, with four questions: **(Q1)** for an equal interaction budget, which method produces the fastest driver? **(Q2)** what do we gain by querying the expert online rather than imitating its demonstrations, and what is the effect of DAgger's mixing coefficient β_i? **(Q3)** what does RLPD owe to symmetric sampling? **(Q4)** can a method outperform the expert?

2 Methods

Behavioral cloning and DAgger. Behavioral cloning (BC) learns a deterministic policy π_θ by regression on the expert's pairs (s_t, a_t), minimizing $\mathcal{L}(\theta) = \sum_t \|a_t - \pi_\theta(s_t)\|^2$. It is trained on the expert's states but deployed on its own: a small error leads it to states absent from the data, and the gap to the expert can grow as $O(H^2\varepsilon)$ over a horizon H [8]. DAgger corrects this distribution shift. At iteration i, we drive with the mixture $\pi_i = \beta_i\pi^* + (1-\beta_i)\hat{\pi}_i$ (the expert acts with probability β_i), label each visited state with the action $\pi^*(s)$, add it to $\mathcal{D}$, then retrain $\hat{\pi}_{i+1}$ on the whole of $\mathcal{D}$. The learner thus learns on its own state distribution, and the gap becomes linear in H. We use $\beta_i = p^i$ and compare $p = 0$, where the expert no longer drives after the demonstrations (often the best choice according to Ross et al. [9]), with $p = 0.5$.

RLPD and symmetric sampling. RLPD is a SAC [4] that is given a fixed set of transitions $\mathcal{D}_{\text{off}}$ in addition to the online buffer $\mathcal{D}_{\text{on}}$. Symmetric sampling builds each batch of size b from $b/2$ transitions of $\mathcal{D}_{\text{off}}$ and $b/2$ of $\mathcal{D}_{\text{on}}$. If the demonstrations were simply poured into the buffer, their share, $|\mathcal{D}_{\text{off}}|/(|\mathcal{D}_{\text{off}}| + |\mathcal{D}_{\text{on}}|)$, would drop below 2% by the end of training here. This mixing is possible because the critic's target does not depend on the policy that produced the transition: RLPD does not imitate the expert, it learns the value of its actions. RLPD adds Layer Normalization to the critics, which bounds their values far from the data, and a high update-to-data ratio (UTD). To fit our compute budget, we use two critics instead of an ensemble of ten, and a UTD of 10 instead of 20. An ablation, *SAC only*, removes the demonstrations.

Conceptual comparison. DAgger receives an action label at every state: a rich signal, but one that only says what the expert would do, and that costs one expert query per step. RLPD only receives a scalar reward, propagated by the Bellman equation: a poorer signal, but one that says what is good. DAgger is therefore

capped by the expert, whereas RLPD can surpass it, at the cost of exploration and a critic that are harder to control.

3 Experimental protocol

Environment. `pystk2-gymnasium` [7] (`supertuxkart/full-v0`), track `cocoa_temple`, one kart, one lap, at most 1500 steps of 0.1 s. With a single kart, the native reward is $r_{t+1} = (\delta_{t+1} - \delta_t)/10 - 0.1 + 10 \cdot \mathbf{1}[\text{finish}]$, where δ_t is the distance travelled. The return of a lap of length $L \approx 1517$ m completed in T steps is therefore

$$G = L/10 - 0.1\,T + 10, \tag{1}$$

and that of an unfinished lap is $G = \delta/10 - 150$: maximizing G is exactly equivalent to minimizing the lap time. The environment is deterministic: a given deterministic policy always drives the same lap (only graphical rendering, never used during evaluation, alters the simulation).

Observations, actions and expert. The learner receives 28 numbers expressed in the kart's frame: velocity, vector and distance to the track centre, maximum steering angle, endpoints and widths of the next 5 track segments. It acts with $a = (\text{steer}, \text{throttle}) \in [-1, 1]^2$, a negative throttle meaning braking; drift, nitro, firing and rescue are disabled. The expert is `Agent1` (team 1 of `pystk_project` [10]), a hand-written controller, from which we remove item and nitro handling and whose actions we project onto the learner's action space. The learner can thus, in principle, reproduce the expert exactly, which is the condition of DAgger's guarantee.

Data. One lap of the expert, i.e. 609 transitions. Since the environment is deterministic, further laps would be copies. We do not add noise to the expert's actions: this technique (DART [6]) itself corrects BC's distribution shift, and would bias the comparison with DAgger. This dataset serves as the initial $\mathcal{D}$ for BC and DAgger, and as $\mathcal{D}_{\text{off}}$ for RLPD.

Evaluation and statistics. Every policy is evaluated on one lap, without noise (mean action for SAC). Determinism makes repeating this episode unnecessary: all variability comes from training. The main metric is the return (1), complemented by the share of finished laps and the lap time. Each method is trained with 10 seeds, which are the statistical unit; a seed's score is that of its final policy, without selecting the best one. The common axis is the number of online environment steps: about 12 000 for DAgger and 30 000 for RLPD, whose score we also read at 12 000 steps to compare at equal budget. We report the mean with a 95 % bootstrap confidence interval, the median and the IQM [1]. Methods are compared with Welch and Mann–Whitney tests [3], and with the (deterministic) expert with a one-sample Student test and a Wilcoxon test. All comparisons, fixed in advance, are corrected with the Holm method [5].

Implementation. PyTorch code without any RL library, trained on the CPU of an Apple M1; hyper-parameters are given in Table 1.

Table 1: Hyper-parameters. Those of SAC are the usual values, not tuned.

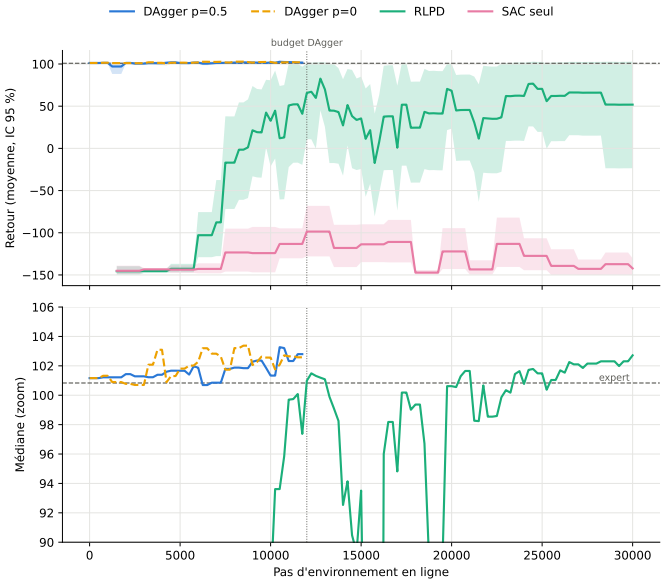
Common	networks	2 hidden layers of 256, ReLU; inputs standardized on the demonstrations
BC and DAgger	training	squared error, tanh output, Adam (10^{-3}), batch 256, 2000 steps
	DAgger	20 iterations of one episode, full retraining, $p \in \{0, 0.5\}$
SAC and RLPD	optimization	Adam ($3 \cdot 10^{-4}$), $\gamma = 0.99$, $\tau = 0.005$, batch 256 (RLPD: 128 + 128)
	critics	2, minimum in the target, Layer Normalization, UTD 10
	actor	tanh-squashed Gaussian, $\alpha_0 = 1$, target entropy -2
	budget	30 000 steps, evaluation about every 1000 steps

4 Results

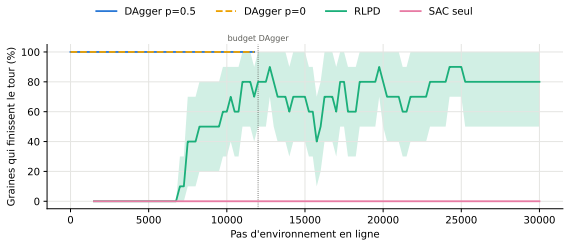
Table 2 summarizes the final scores, and Fig. 1 shows how each method gets there as a function of environment steps. The expert is the reference: a return of 100.83, i.e. 60.9 s.

Table 2: Final scores over 10 seeds (one deterministic evaluation per seed). Return: mean and 95 % bootstrap CI. Success: seeds that finish the lap. Time: mean over finished laps. 1st lap: median number of online steps before the first finished lap. Last 5: mean of the last 5 evaluations of each seed (exploratory analysis). RLPD @12k: RLPD read at DAgger's budget.

Method	Return [95 % CI]	Median	IQM	Success	Time (s)	1st lap (steps)	Last 5 [95 % CI]
Expert	100.83	–	–	100 %	60.9	–	–
BC	101.20 [101.03, 101.40]	101.16	101.17	100 %	60.5	0	–
DAgger p=0	90.67 [68.38, 102.22]	101.64	101.45	90 %	60.0	0	99.81 [95.37, 102.29]
DAgger p=0.5	102.13 [101.02, 103.11]	102.88	102.51	100 %	59.6	0	102.11 [101.57, 102.59]
RLPD @12k	65.72 [18.58, 100.93]	100.98	98.70	80 %	62.2	–	–
RLPD	51.84 [−23.55, 102.76]	102.71	101.84	80 %	59.4	8596	56.91 [−8.53, 102.45]
SAC only	−142.22 [−149.93, −130.25]	−149.97	−148.70	0 %	–	– (10 never)	−139.88 [−147.14, −131.57]



(a) Return during training. Top: mean and 95 % CI. Bottom: median, zoomed around the expert (dashed).



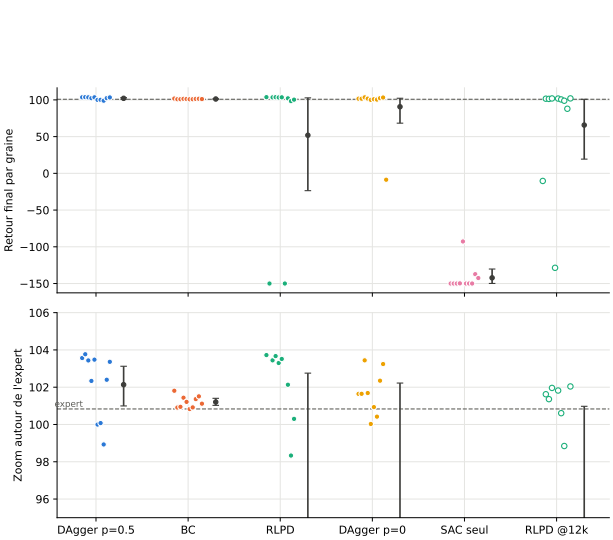
(b) Share of seeds whose current policy finishes the lap (mean and 95 % CI).

Figure 1: Learning curves as a function of the number of online environment steps (10 seeds per method). The vertical dotted line marks DAgger's budget. Both DAgger variants stay at the expert's level from the start; RLPD only learns to drive after several thousand steps, then remains unstable; SAC only never succeeds.

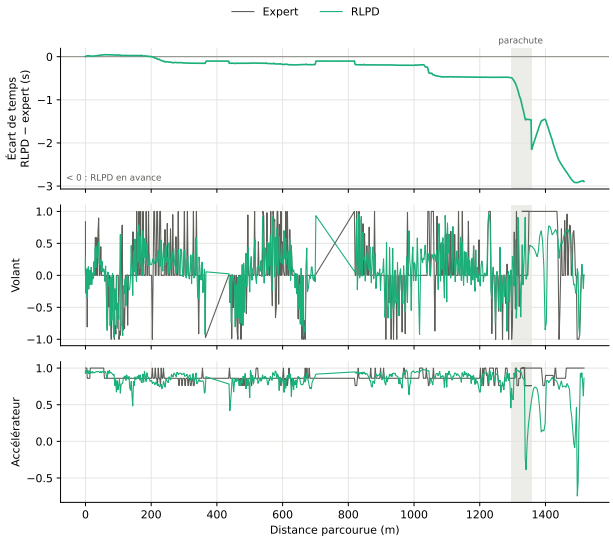
DAgger: at the expert's level right away. BC, which is DAgger's iteration 0, already finishes all its laps and slightly outperforms the expert (101.20 versus 100.83). The following DAgger iterations stay at this level (Fig. 1a) and finish 209 laps out of 210 evaluations for $p = 0$, and 210 out of 210 for $p = 0.5$. With $p = 0.5$, DAgger obtains the best scores of the study: mean 102.13, median 102.88, IQM 102.51, i.e. about 1.3 s faster than the expert. The $p = 0$ variant has a similar median (101.64), but its last iteration failed on one seed (kart stuck 100 m from the finish line), which drops its mean to 90.67. From one iteration to the next, the lap time varies between 58 and 66 s: each retraining starts from a fresh network, and a small steering difference can change the whole end of the lap.

RLPD: late and unstable learning. RLPD starts with a random actor and stays still for several thousand steps, while the temperature α decreases and the critic learns the value of the expert's actions. All seeds then complete a first lap, between 6 800 and 12 500 steps (median 8 600). An unstable phase follows, visible on the median between 12 000 and 20 000 steps, where policies alternate between fast laps and failures (Fig. 1b). At the end of training, 8 seeds out of 10 finish the lap, in 59.4 s on average, and 5 of them in under 58.5 s; the other two have collapsed and stay still. Hence a high median (102.71), close to DAgger, but a mean of 51.84.

Without demonstrations, SAC fails. With the same algorithm and the same budget, but without expert data, no seed finishes a single lap: the best kart covers 1 316 m out of 1 517, and most end up still (final return −142.22).



(a) Final score of each seed (dots) and mean with 95 % CI (bars). Bottom: zoom around the expert.



(b) RLPD (seed 0, 58.0 s) versus the expert, as a function of the distance travelled: time gap, steering and throttle. The grey area is where the expert gets the parachute.

Figure 2: Left: the spread of final scores reveals RLPD's collapses and the single failure of DAgger $p = 0$. Right: RLPD's lead over the expert builds up entirely at the banana.

Beating the expert by avoiding a banana. The expert drives over the same banana on every lap and gets a parachute for 3.8 s (38 steps). The fastest DAgger and RLPD policies (58.0 to 58.3 s) take another line at this spot and never hit the banana. Figure 2b shows it for RLPD: the time gap stays almost zero for 1 300 m, then widens by about 2.9 s precisely in this area. BC, on the other hand, gets the parachute like the expert; its slight advantage (0.4 s on average) comes from the end of the lap, where it drives more smoothly than the expert's threshold-based rules.

Statistical tests. Table 3 gives all the comparisons fixed in advance. After Holm correction over the 14 tests, only two differences are significant: SAC only is worse than RLPD (-194, $p = 0.003$ for Welch) and than the expert. BC outperforms the expert by 0.37 with a confidence interval that excludes 0, but the correction leaves it at the edge of the threshold ($p = 0.05$ to 0.06). The other differences, in particular between DAgger and RLPD, are not significant: the intervals are very wide because of RLPD's collapses, and 10 seeds can only detect differences of about 1.25 standard deviations.

5 Discussion

Q1 – At equal budget, DAgger wins. After about 12 000 steps, DAgger finishes all its laps, whereas RLPD only finishes 8 out of 10 and has an IQM almost 4 points lower (98.70 versus 102.51). The confidence interval of the difference of means excludes 0 (-36.41 [-83.47, -1.34]), even though the test does not survive the Holm correction. DAgger is also much cheaper in compute: a few minutes per seed, versus about 40 minutes for RLPD with 10 critic updates per step. In return, it requires an expert that can be queried at every step, i.e. about 12 600 labels, whereas RLPD makes do with a single recorded lap. With 2.5 times more steps, RLPD catches up with DAgger in median, but not in reliability.

Q2 – Querying the expert online brings little here. DAgger does not do significantly better than BC (+0.93, CI [-0.20, 1.94]). Theory explains why: BC's $O(H^2\varepsilon)$ bound describes the worst case, where every error pushes the learner away from the expert's trajectory. Here, the environment is deterministic, the expert only reacts to the current state and the learner can reproduce it exactly: imitation is accurate enough that the learner never leaves the trajectory, and the distribution shift does not appear. The choice of β_i has no demonstrable effect; the $p = 0.5$ variant had no failure, but a single seed separates the two variants.

Q3 – Symmetric sampling is essential. This is the clearest effect of the study. The expert's 609 transitions are enough to take SAC from zero finished laps to ten seeds out of ten that finish at least one. Since they make

Table 3: Comparisons between methods and with the expert: difference of means (Δ, with 95 % bootstrap CI) and p-values corrected with the Holm method over all 14 tests. Against the deterministic expert, the tests are one-sample (Student and Wilcoxon).

Comparison	Δ [95 % CI]	p Welch (Holm)	p Mann–Whitney (Holm)
RLPD − DAgger p=0.5	−50.29 [−125.51, 0.76]	1.000	1.000
RLPD − DAgger p=0	−38.83 [−113.71, 21.96]	1.000	1.000
RLPD @12k − DAgger p=0.5	−36.41 [−83.47, −1.34]	1.000	0.190
RLPD @12k − DAgger p=0	−24.95 [−80.62, 20.30]	1.000	1.000
SAC only − RLPD	−194.06 [−249.47, −122.38]	0.003	0.092
DAgger p=0.5 − DAgger p=0	11.46 [−0.55, 34.06]	1.000	1.000
DAgger p=0 − BC	−10.53 [−32.88, 1.03]	1.000	1.000
DAgger p=0.5 − BC	0.93 [−0.20, 1.94]	1.000	1.000
DAgger p=0.5 − Expert	1.30 [0.20, 2.28]	0.520	0.488
BC − Expert	0.37 [0.19, 0.57]	0.062	0.051
RLPD − Expert	−48.99 [−124.36, 1.92]	1.000	1.000
DAgger p=0 − Expert	−10.16 [−32.45, 1.40]	1.000	1.000
SAC only − Expert	−243.05 [−250.76, −231.08]	0.000	0.027
RLPD @12k − Expert	−35.11 [−82.72, 0.05]	1.000	1.000

up half of every batch, the critic learns from the start the value of a trajectory that reaches the finish line, which random exploration does not discover in 30 000 steps.

Q4 – Both methods can outperform the expert, for different reasons. RLPD does so as expected, by directly optimizing the lap time. The case of DAgger is more surprising: DAgger's guarantee bears on the imitation error, not on performance, and nothing prevents the learner from doing better than its teacher. By distilling a rule-based controller into a neural network, DAgger obtains smoother driving, which sometimes finds a line that avoids the banana.

Limitations. We simplified RLPD (2 critics instead of 10, UTD of 10 instead of 20) to fit our compute budget. The critic ensemble is precisely there to limit the overestimation and overfitting caused by a high UTD: our two collapses are probably related to this simplification, and testing the full RLPD is the natural next step of this work. The study covers a single track and a single expert, with 10 seeds per method. The last-iteration score is noisy for DAgger, as confirmed by the more stable mean of the last 5 evaluations (102.11 for $p = 0.5$). Finally, the reward rewards distance travelled: two intermediate RLPD evaluations measured more distance than the track length, which inflates their return beyond the theoretical maximum. No final policy is affected, and the lap time remains the reliable measure in that case.

6 Conclusion

On this deterministic task, with a reactive and reproducible expert, DAgger is the most efficient and most reliable method: it reaches and slightly surpasses the expert from the start, at minimal compute cost, provided the expert can be queried at every step. RLPD only needs one recorded lap and can also surpass the expert, but it requires more interactions, much more compute, and remains unstable in our simplified version. Symmetric sampling is the cornerstone of RLPD: without it, the same algorithm does not learn to complete a lap.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[2] Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning (ICML)*, 2023.

[3] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. A hitchhiker's guide to statistical comparisons of reinforcement learning algorithms. *arXiv preprint arXiv:1904.06979*, 2019.

[4] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

[5] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6 (2):65–70, 1979.

[6] Michael Laskey, Jonathan Lee, Roy Fox, Anca Dragan, and Ken Goldberg. DART: Noise injection for robust imitation learning. In *Conference on Robot Learning (CoRL)*, 2017.

[7] Benjamin Piwowarski. pystk2-gymnasium: Gymnasium wrapper for PySTK2. https://github.com/bpiwowar/pystk2-gymnasium.

[8] Stéphane Ross and J. Andrew Bagnell. Efficient reductions for imitation learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.

[9] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[10] Olivier Sigaud. pystk_project: agents for SuperTuxKart. https://github.com/osigaud/pystk_project.